

— SOLUTIONS ENGINEERING

Agentic AI workflows

Coordinating a distributed AI agent swarm on the event backbone the customer already runs

What an agent is

- Unlike an LLM on its own, an agent can **take actions** — it's a wrapper around an LLM with tools, instructions, and a knowledge base
- Every call to the LLM carries the tools and instructions; the LLM replies with which tool to run; the agent runs it, feeds the result back, and the loop repeats until the task is done
- **Isolation matters:** too many tools in one agent overwhelm the context and cause hallucinations
- Each agent should run in its own **sandbox** — a compromised agent must not reach another's data or credentials

A P&C carrier whose second-largest controllable cost is deciding claims

- Property & Casualty; millions of dollars in claims a year at very high volume
- Today a claim sits with an adjuster who reads hundreds of pages, checks the policy language by hand, and researches similar past claims — **days per claim**, and two adjusters can reach different answers on the same facts
- The cost of deciding claims — adjuster time, appraisers, reviews — is their largest controllable expense after the claim payouts themselves
- **Board mandate:** cut claims cycle time and cost without adding headcount — and every automated decision has to be defensible to a regulator

→ The ask isn't "add an LLM" — it's coordinating many specialists per claim, at volume, defensibly.

What a solution has to do

- **Many specialists per claim, in parallel:** claim data, policy coverage, fraud signal, medical review — then one adjudication step
- **Elastic:** a catastrophe event can 5–10× daily claim volume for weeks
- **Runs where the data is:** some agents on-prem (PII / PHI stays inside the boundary), some in cloud
- **Every input and decision retained and replayable** — "why was claim 4471 denied?" answered from a record, not reconstructed
- **Trustworthy decisions:** defend against prompt injection and poisoned claim documents; catch model drift as policies and the model change
- **Agents built in-house, each isolated, no runtime lock-in**

Hyperscaler agent platforms

Managed agents

FOUNDRY AGENT SERVICE · BEDROCK AGENTCORE

- PRO** Runs on cloud infra with micro-VM sandboxes for isolation
- PRO** Very low code and operational overhead
- CON** Black box and vendor lock-in
- CON** Severe regional capacity limits — pushing customers to run workloads outside their primary business region

Self-hosted agents

AGENT FRAMEWORK, RUN YOURSELF

- PRO** Full control over what the agent does
- PRO** Fine when the use case is simple
- CON** You lose sandbox isolation and the other managed benefits
- CON** The workflow API doesn't make it run distributed
github.com/rahulbats/workflow-skills-agent-framework-demo

Orchestration frameworks — LangGraph, Temporal

- PRO** Strong open-source community
- PRO** Workflows are distributed
- CON** Built around communication **between** agents — the workers have to know about and report back to the orchestrator / manager

The coordination is point-to-point, and the workflow history lives inside one engine — no fraud team, BI, or model-retraining pipeline can subscribe to it.

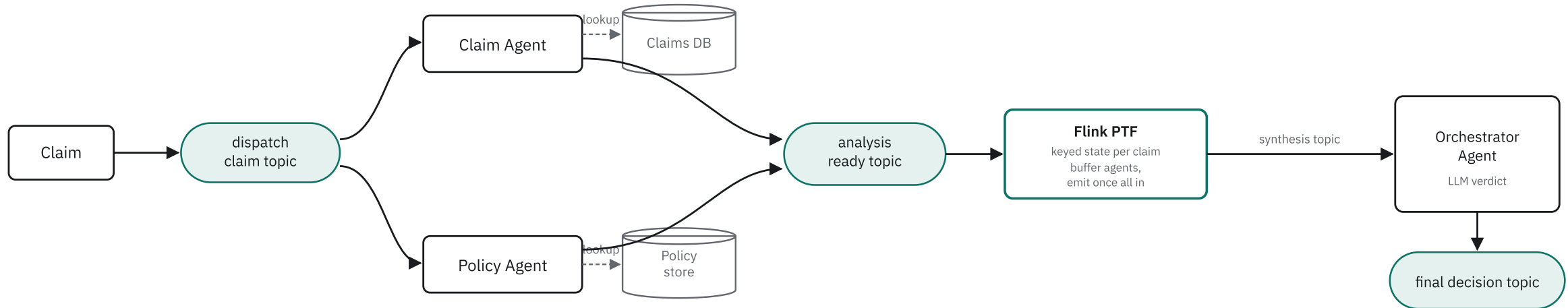
→ The gap all three share: the coordination layer is bespoke, coupled, and **not** the system of record.

Kafka as the agents' nervous system

- The customer already runs Kafka for mission-critical event flows — extending it to agent coordination adds **no new system of record**
- Tasks are dropped on topics; a distributed swarm of **fully isolated** agents on containers / VMs picks them up in parallel
- An agent or service crash simply **resumes from the last committed offset** — no lost work, no bespoke retry logic
- Fan-in is a Flink **ProcessTableFunction** (PTF, Flink 2.1) — keyed state buffers each agent's result and emits one record per claim once all report; the orchestrator runs the final LLM verdict from that
- **No point-to-point calls** — agents never talk to each other or the orchestrator; the log is the only thing they share, and every result also feeds fraud analytics, ops, and retraining

— HOW IT WORKS

One claim, end to end



Agents never call each other or the orchestrator — they only produce to and consume from topics. Flink holds the only cross-agent state (the per-claim barrier) and emits once all agents report.

Each requirement maps to a property of the design

REQUIREMENT HOW THE DESIGN DELIVERS IT

Specialists in parallel → one verdict	Agents are independent Kafka consumers; a Flink ProcessTableFunction does the fan-in barrier in keyed state — wait for all N, then emit once.
Elastic to 5–10× for CAT events	KEDA scales each agent on consumer lag — no pre-provisioning, no shared-quota wall the way a managed tier has one.
Runs where the data is	Agents are the customer's containers; Kafka spans on-prem and cloud; PII topics stay on the on-prem cluster.
Auditable	The log is the record: every agent input, output, and decision — schema-governed, replayable, and visible in Stream Lineage.
Trustworthy decisions	Guardrails (input scanning, output validation, confidence gates) are just more functions in the pipeline; low-confidence claims route to a review topic. A new model is evaluated by replaying the retained history and diffing the outcomes.
In-house agents, no lock-in	The customer's code and model choice; the substrate is portable across clouds and on-prem.

→ "Why was claim 4471 denied?" — replay the `claim_id` and read every agent's exact input and output.

— LIVE — A SCREEN RECORDING IS READY IF THE NETWORK ISN'T

Demo

- Confluent Cloud Kafka + Schema Registry + managed Flink
- Two agents, one consumer group each, against a local model
(Qwen 3, 4-bit quantized)
- Barrier: a Flink **ProcessTableFunction** partitioned by claim — emits one record once both agents report
- Orchestrator consumes that record, runs the LLM, produces the decision

WHAT YOU'LL SEE, ACROSS THREE TERMINALS

```
dispatch CLM-1001 — burst pipe, $8,200
claim agent → retrieves the record
policy agent → retrieves POL-55 (H0-3)
Flink → emits 1 record, both findings
orchestrator → COVERED
$8,200 — $1,000 deductible = $7,200 payable
cites SEC-II-A (sudden & accidental discharge)
```

→ Then a flood claim on a policy with no flood endorsement — same pipeline, comes back **NOT COVERED** .

Summary & next steps

THE RECOMMENDATION

- Coordinate the swarm on the **event backbone you already run** — not a new engine
- Agents stay yours: your code, your models, each isolated
- Flink does the fan-in; the log is the audit trail
- Distributed, elastic, and explainable at once

NEXT STEPS

- A two-week **scoping workshop** — one experienced adjuster, a non-production claims feed — to put real numbers on cycle time and cost for one claim type
- **Phase 1: shadow mode** — the swarm decides alongside your adjusters, nobody acts on it; that comparison is also the first eval baseline
- Before go-live: a **guardrail layer** (injection / PII in, decision validation out) and an **eval harness** that replays claims across model versions to catch drift
- Adjuster corrections land on their own topic — a **labeled dataset that grows itself**, feeding periodic fine-tuning
- Then: an **agent-spec.yaml** and a K8s operator stand the swarm up; KEDA scales it on lag

→ One decision this week: do we book the workshop?